



LIUM

Laboratoire d'Informatique
Le Mans Université



**Le Mans
Université**

Projet de Recherche M2 - Ingénierie R&D

Amélioration de la prononciation des modèles TTS de type Flow-Matching conditionnel

Présenté par :

Naif ASSWIEL

Yahya BAHLOUL

Supervisé par:

M. Aghilas Sini

M. Kevin Vythelingum

Sommaire

- Voxygen.
- Contexte et problématique.
- Jeu de données.
- Approche 1 : Adaptation par Fine-tuning.
- Approche 2 : Optimisation par Apprentissage par Renforcement.
- Conclusion.

Voxygen

- **Fondé** en 2011, **basé** en Bretagne.
- **Objectif:** Donner une voix naturelle et expressive aux machines (Text to speech).
- **Applications concrètes:**
 - A- l'ajout de voix à des chatbots ou assistants.
 - B- Automatiser les réponses vocales de centres d'appels.
- **Client:** SNCF, EDF.



Contexte et problématique

Problèmes:



- 1- Le défi de la langue (Biais vers l'anglais).
- 2- La sensibilité au "Prompt" (One-Shot).

Impacts:



- 1- **Sur la langue** : Génération d'un accent étranger et manque de naturel en français.
- 2- **Sur le One-Shot** : Variation critique de la qualité selon le fichier audio choisi en référence.

Objective:



- 1- Affiner (Fine-tuning) le modèle sur des données françaises.
- 2 - Développer une approche non supervisée pour sélectionner automatiquement le prompt optimal.

Jeu de données

Caractéristiques :



- **Source** : Blizzard2023 (Audiobook / Parole lue).
- **Langue** : Français (Haute qualité studio).
- **Volume** : ~53 heures d'enregistrement.
- **Locutrices** :
 - **NEB** : ~51h (63 707 segments).
 - **AD** : ~2h (2 041 segments).

Prétraitement :



- **Segmentation** : Découpage en extraits alignés de 3 à 5 secondes.
- **Nettoyage Textuel** : Suppression des caractères non conformes (~, ^, -, \$).
- **Formatage** : Structuration pour ZipVoice (Wav + Prompt + Texte)

Approche 1:

Adaptation par Fine-tuning

Architecture et Choix Technologiques : Justifications

Composant	Choix Retenu	Alternative	Argument Clé (Le "Pourquoi")
Paradigme Génératif	Flow Matching	Diffusion	Efficacité Géométrique : Trajectoires droites (OT) ➡ Inférence plus rapide et stable que les chemins stochastiques de la diffusion.
Modèle TTS	ZipVoice	F5-TTS	Maîtrise & Modularité : ZipVoice offre une architecture (DiT) plus flexible pour la recherche expérimentale. F5-TTS, bien que performant, est une architecture plus massive et moins malléable pour nos tests d'optimisation.
Métrique / ASR	Whisper V3	Wav2Vec 2.0	Robustesse Multilingue : Entraîné sur 680k heures (supervisé), Whisper gère mieux le bruit et la ponctuation que Wav2Vec (auto-supervisé) pour calculer un WER fiable.
Espace Latent (Agent)	SONAR	BERT / Wav2Vec	Alignement Multimodal : SONAR place le texte et l'audio dans le <i>même</i> espace vectoriel, indispensable pour que l'agent compare Texte Cible et Prompt Audio.

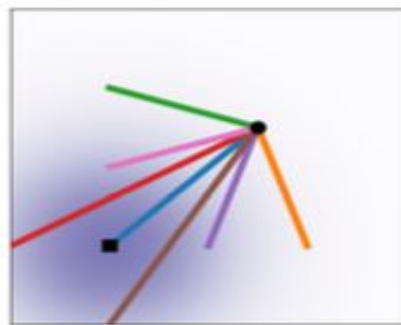
Le Paradigme Flow Matching

Le Flow Matching permet d'apprendre une transformation directe du bruit vers la donnée.

- **Transport Optimal (OT)** : Impose une trajectoire rectiligne (vs stochastique pour la Diffusion).



Diffusion



OT

Trajectoire plus simple = Apprentissage stable + Inférence rapide (NFE réduit)

Configuration	Mesure : Whisper Base		Mesure : Whisper FT LoRA	
	CER	WER	CER	WER
Référence (Ground Truth)				
AD	0.0798	0.2272	0.0072	0.0257
NEB	0.1219	0.3103	0.0077	0.0265
ZipVoice (Pré-entraîné)				
AD	0.2802	0.6426	0.1484	0.3194
NEB	0.4914	0.8827	0.1159	0.2668
ZipVoice FT V1 (Mixte - 50k it)				
AD	0.3382	0.7635	0.2928	0.6529
NEB	0.3413	0.7567	0.2795	0.6390
ZipVoice FT V2 (NEB - 10k it)				
NEB	0.0804	0.1923	0.0690	0.1784

Fiabilisation de la chaîne d'évaluation

Le Protocole de Mesure

Outil : `tts4all_eval` (LIUM).

Moteur de transcription : Whisper Large-v3.

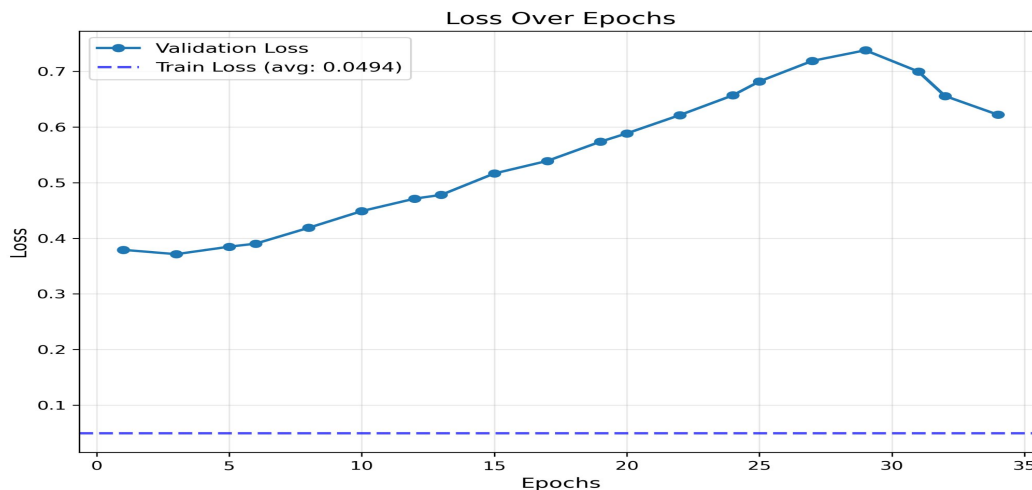
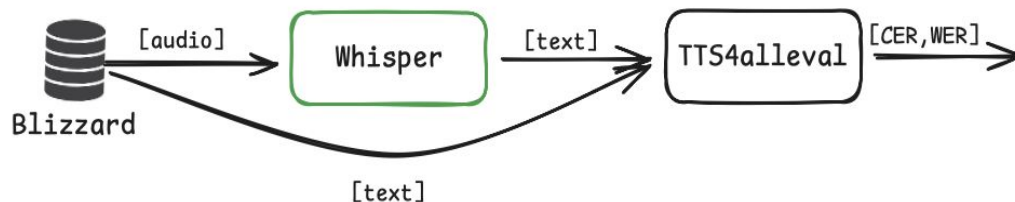
Échec de la Première Approche

Stratégie V1 : *Full Fine-Tuning* .

Résultat : WER : > 60 %.

Diagnostics : Oubli Catastrophique

Configuration	Mesure : Whisper Base	
	CER	WER
Référence (Ground Truth)		
AD	0.0798	0.2272
NEB	0.1219	0.3103



Validation de la métrique : Succès de LoRA

La Stratégie Corrective (LoRA)

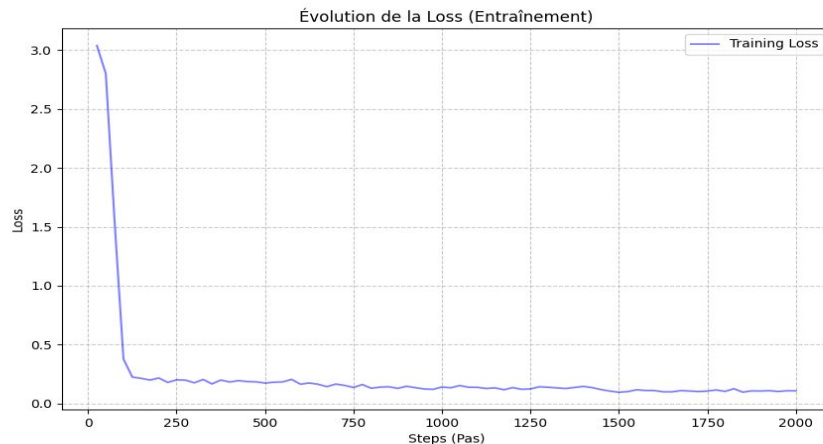
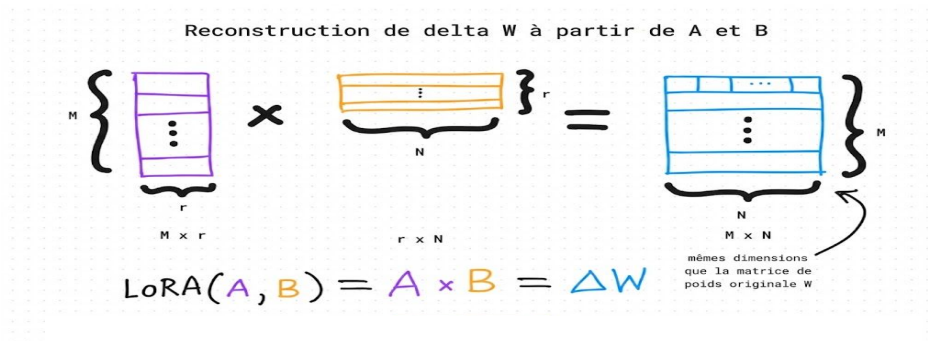
Principe : Adaptation de bas rang.

- Gel des 1,5 milliard de paramètres
- Injection de matrices

Configuration : Rang (r) : 32 / Alpha (α) : 64

Comparaison des taux d'erreur (WER) sur la référence :

Configuration	Mesure : Whisper Base		Mesure : Whisper FT LoRA	
	CER	WER	CER	WER
Référence (Ground Truth)				
AD	0.0798	0.2272	0.0072	0.0257
NEB	0.1219	0.3103	0.0077	0.0265



Adaptation du modèle de synthèse (ZipVoice)

Le Problème de ZipVoice

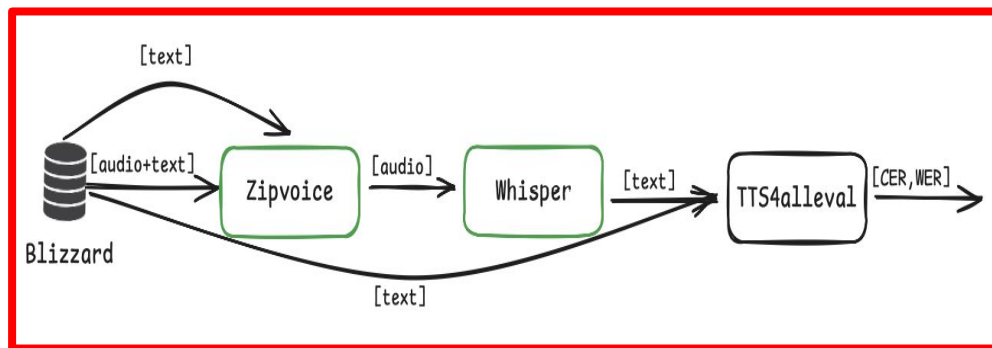
État initial : Modèle pré-entraîné sur le dataset **Emilia** (101k heures).

Le Biais : Données majoritairement Anglais/Chinois (95 %).

Performance Baseline (Zero-Shot) :

- **WER : 31,94 %** (Taux d'erreur élevé).
- **Conséquence** : Accent étranger marqué et instabilité du timbre.

Stratégie : Fine-tuning supervisé pour spécialiser le modèle sur la phonétique française.



ZipVoice (Pré-entraîné)				
AD	0.2802	0.6426	0.1484	0.3194
NEB	0.4914	0.8827	0.1159	0.2668

Fine-tuning ZipVoice : Limites de l'approche multi-locuteurs

Configuration Expérimentale (V1)

- **Stratégie** : Entraînement "Mixte".
- **Durée** : 50 000 itérations.
- **Hypothèse** : Le modèle apprendra à généraliser et à distinguer les deux timbres.

Résultats : WER V1 : 65,29 % (vs Baseline 31,94 %).

Diagnostic technique :

1. **Incohérence Timbrale**
2. **Sur-apprentissage du bruit**
3. **Oubli Catastrophique**

Succès de la spécialisation Mono-Locuteur

La Stratégie Corrective (V2)

- **Nouvelle approche** : Spécialisation stricte.
 - **Données** : Exclusion de la voix masculine (AD). Entraînement uniquement sur la locutrice cible (NEB).
- **Ajustement des Hyperparamètres** :
 - **Learning Rate** : Réduit à **10⁻⁶** (Stabilisation).
 - **Early Stopping** : Arrêt précoce à **10 000 itérations** (vs 50k pour la V1).
- **Objectif technique** :
 - Adapter le timbre en douceur sans détruire la "grammaire" acoustique du modèle pré-entraîné.

Résultats Comparatifs (La Preuve)

- **Évolution du Taux d'Erreur (WER)** :
 - *ZipVoice Base (Zero-Shot)* : **31,94 %** (Point de départ).
 - *ZipVoice V1 (Échec)* : **65,29 %** (Régression).
 - **ZipVoice V2 (Succès)** : **17,84 %**.
- **Bilan qualitatif** :
 - Gain de **~14 points** par rapport au modèle original.
 - Restauration de l'intelligibilité et fidélité du timbre.
 - *Limite observée* : Quelques hallucinations musicales sur les phrases très courtes.

Approche2:

**Optimisation de sélection de prompt par Apprentissage
par Renforcement**

Impact Critique du Prompt (Motivation)

Le Constat (ZipVoice) :



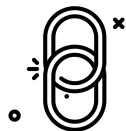
- **Modèle One-Shot** : Voix clonée à partir d'un court audio (Prompt).
- **Problème** : Qualité varie selon le prompt choisi.

Preuve Expérimentale (Nos tests) :



- **Prompt "Idéal"** (Texte identique au prompt) → CER $\approx$ 0.00 (Parfait).
- **Prompt "Aléatoire"** → CER jusqu'à 0.48 (Qualité très dégradée).

Le Verrou Scientifique :



- **Défi** : Identifier le "meilleur" prompt pour un text cible.
- **Contrainte majeure** : Absence de données annotées (Vérité Terrain)

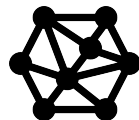
Formulation en Apprentissage par Renforcement

Pourquoi le RL ?



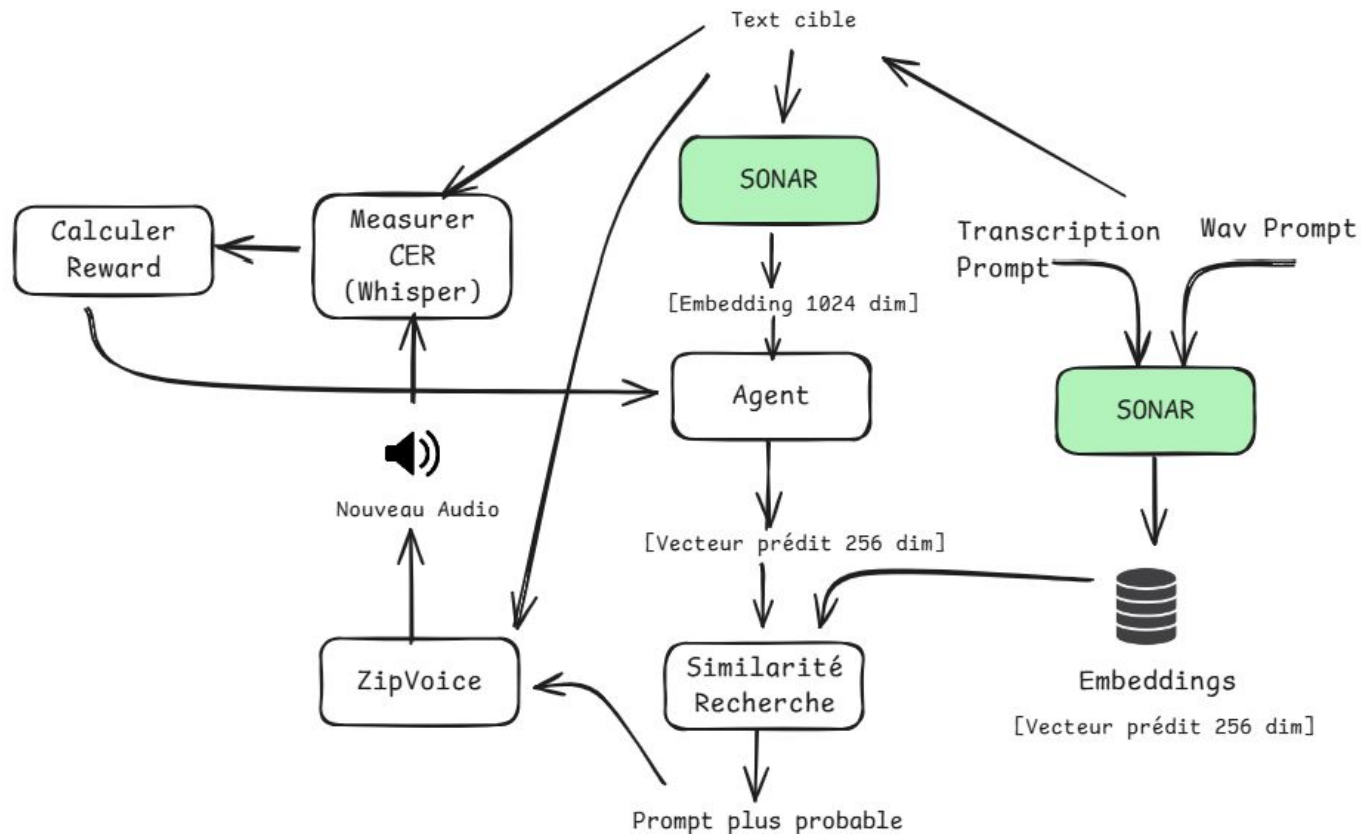
- Permet d'apprendre par "essais-erreurs" sans avoir besoin de connaître la bonne réponse à l'avance.

Modélisation : Contextual Bandit



- **L'Agent** : Le modèle qui choisit le prompt.
- **L'État (State)** : La phrase cible à synthétiser.
- **L'Action** : Le choix d'un prompt dans la base de données.
- **La Récompense (Reward)** : La qualité de l'audio produit.

Architecture du Pipeline d'Apprentissage



Défis rencontrés et Solutions

1- Problème de "Mode Collapse" :



- **Constat** : L'agent s'enfermait à choisir les mêmes 10 prompts sur 63 000.

- **Solution** : Ajouter une Pénalité de Diversité pour forcer l'exploration.

2- Instabilité et Oubli Catastrophique :



- **Constat** : L'apprentissage séquentiel écrasait les connaissances précédentes.

- **Solution** : Ajout d'un Replay Buffer pour stabiliser la convergence et briser les corrélations temporelles.

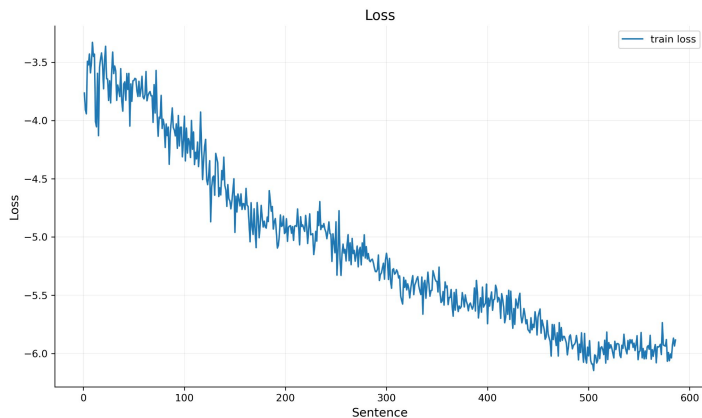
3- Qualité des Représentations:



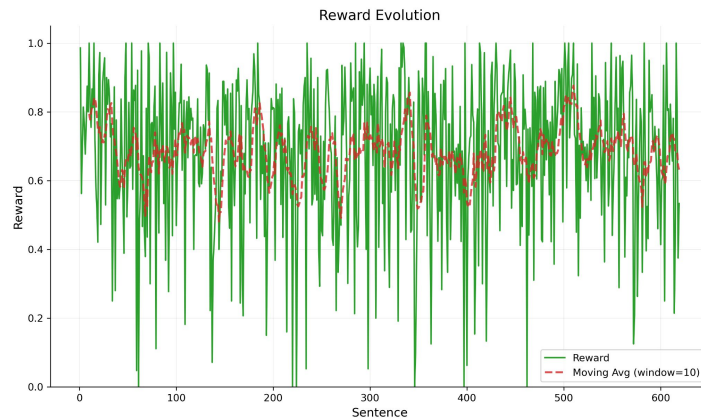
- **Optimisation** : Passage d'embeddings de 100 à 256 dimensions.

Analyse des Résultats

- 1- **Convergence** : Loss diminue et se stabilise après environ 500 phrases.
- 2- **Préférence pour le court** : L'agent sélectionne des prompts courts (≤ 7 mots) 3,5 fois plus souvent. CER moyen (0.29) que les longs (0.40)
- 3 - **Phrases très courtes (1-3 mots)** : Performance dégradée (CER 0.44 vs 0.27) par manque de contexte.



Loss du modèle “baseline”



Évolution de la récompense

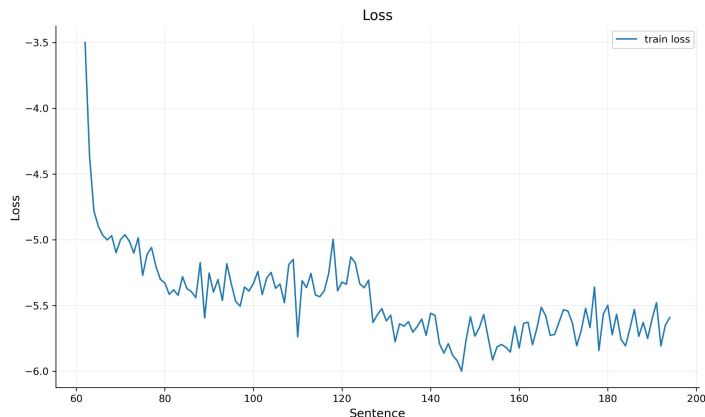
Expérimentations avec

1- **Expérimentation** : Filtrage des prompts (Longueur minimale de 6 mots).

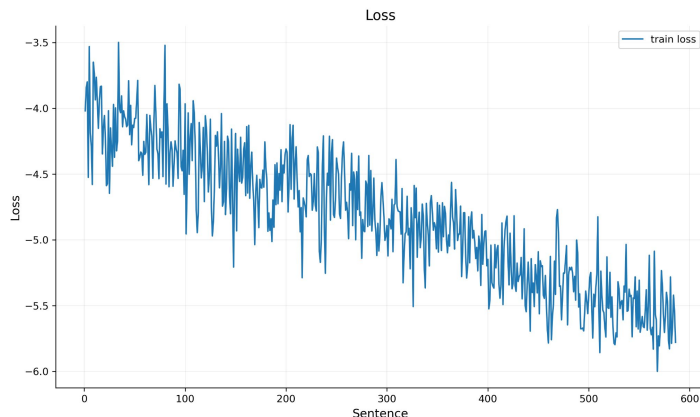
- **Analyse** :

2- **Expérimentation** : Espace de recherche restreint (Réduction de 63k à 5k prompts).

- **Analyse** :



Longueur minimale de 6 mots



Espace Restreint (5k prompts)

Limitations



- **Pipeline chronophage** : ~600 phrases en 48h (SONAR + ZipVoice + Whisper)

Conclusion



1 - Fine-tuning avec LoRA et le plus performants (~3x)

2 - Agent apprend à choisir des prompts courts (≤ 7 mots) sans vérité terrain.

3 - **Bilan Global** : Une piste pour l'optimisation automatique des prompts ("AI Prompt Engineering") dans la synthèse vocale

Des Questions ?

Références:

- <https://arxiv.org/abs/2506.13053>
- <https://arxiv.org/abs/1202.1334>
- <https://zenodo.org/records/7560290>
- <https://arxiv.org/abs/2210.02747>
- <https://codecompass00.substack.com/p/what-is-lora-a-visual-guide-llm-fine-tuning>

Annexe – Analyse des résultats

Métrique	Baseline (260326)	espace de 5 000	longueur min 6 mots
Nb phrases	619	586	194
Reward moyenne	0.686	0.642	0.641
CER moyen	0.316	0.362	0.360
Value Loss finale	0.002	0.010	0.007

Comparaison des résultats de différentes pipelines

Métrique	Valeur
Avg Loss moyen	-5.13
Policy Loss moyen	-6.05
Contrastive Loss	~ 3.33 (faible, stable)
Value Loss	0.005 → 0.002 (réduction 57.7%)
Reward moyen	0.686
CER moyen	0.316

Détails des résultats de la pipeline de base

Annexe – Rôles des losses

- Formule de loss de l'agent :

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{PG}}}_{\text{Politique}} + \underbrace{0.3 \cdot \mathcal{L}_{\text{InfoNCE}}}_{\text{Contraste}} + \underbrace{\mathcal{L}_{\text{value}}}_{\text{Valeur}} - \underbrace{\alpha \cdot H(\text{pred})}_{\text{Entropie}} + \underbrace{\beta \cdot \text{Pen}_{\text{div}}}_{\text{Diversité}}$$

1- La Policy Loss

- **Cible**: l'acteur
- **Rôle** : Elle apprend à l'acteur à choisir la direction gagnante.

2- La Contrastive Loss:

- **Cible** : l'acteur (Structure)
- **Rôle** : Elle apprend à l'acteur à structurer l'espace latent et à placer les vecteurs "gagnants" loin des vecteurs "perdants".

3- La Value Loss:

- **Cible** : le critique (Comportement)
- **Rôle** : elle apprend à prédire la récompense future.

4- Entropie: ($\alpha = 0,1$)

- **Role** : empêcher le "Policy Collapse" (force la diversité **statistiquement**).
- Pred vecteur 256D produit par l'Acteur.

5- Pénalité de Diversité: ($\beta = 0,1$)

- forcer géométriquement les prédictions d'un même batch à s'éloigner les unes des autres pour garantir la diversité
- Garder le vecteur d'un batch orthogonaux pour la diversité

Annexe – Architecture de l'agent

1 - L'Acteur :

Input: l'état (texte cible 1024D)

Output: l'action optimale (vecteur prompt 256D)

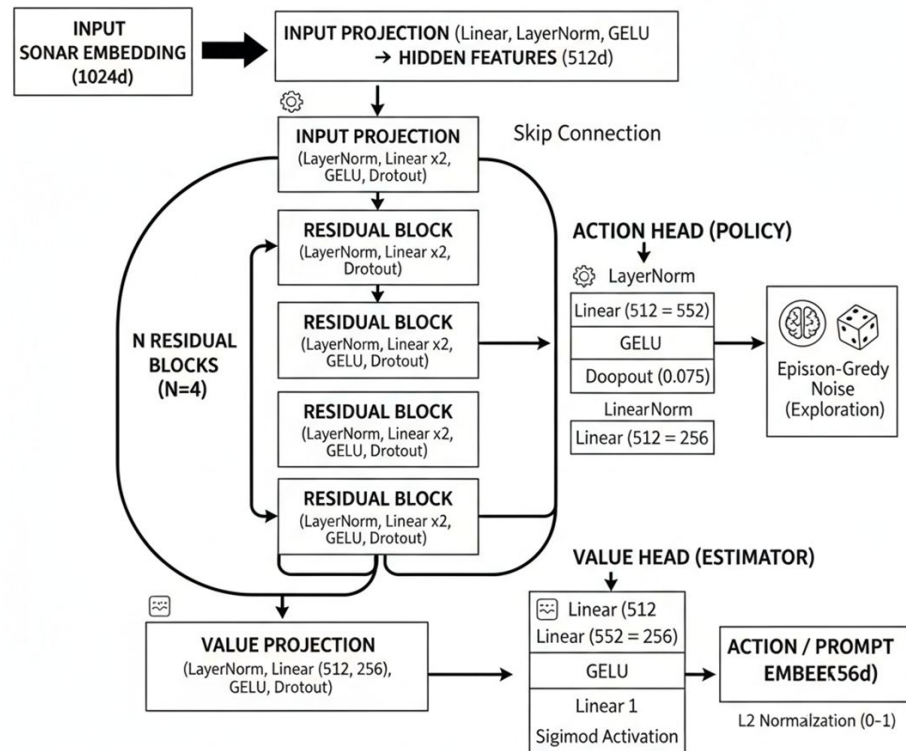
Objective: pour maximiser la récompense.

2 - Le Critique :

Input: l'état (texte cible 1024D)

Output: la valeur estimée ($V(s)$)

Objective: minimiser l'erreur de prédiction de la récompense



Annexe – loss policy

- Loss négative (Normal): la composante dominante est la policy loss, est définie par :

$$L_{\text{policy}} = -\frac{1}{B} \sum_i \log(\cos(\hat{a}_i, a_i)) \cdot \hat{A}_i$$

- $\hat{\mathbf{a}}_i$: vecteur 256D prédit,
- $\mathbf{a}_i$ (Action réelle) : vecteur 256D du prompt sélectionnée
- **L'avantage** est la différence entre la récompense obtenue et la valeur prédite (Récom prédite)
donner le **SIGNES (+ ou -) au gradient** pour la Policy Los

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + 10^{-8}}$$

- $\mathbf{r}_i$ (La Récompense de l'action i) : 1- CER
- $\bar{\mathbf{r}}$: la récompense de référence (moyen de récompense réelle de 64 échantillons).
- σ_r : (L'Écart-Type des récompenses référence)

Policy loss (Role) : Entraîne **l'acteur** à maximiser la récompense en augmentant la probabilité des actions ayant obtenu un résultat supérieur aux attentes

Annexe – loss contrastive

Role :

- structurer l'espace latent, Forcer la prédiction de l'agent (p) à ressembler au meilleur prompt disponible et à être très différente des mauvais prompts.
- Séparer les bons et les mauvais choix soient bien séparés

$$L_{\text{InfoNCE}} = -\log \frac{e^{\text{sim}(p, a^+)/\tau}}{e^{\text{sim}(p, a^+)/\tau} + \sum_j e^{\text{sim}(p, a_j^-)/\tau}}$$

- Le “-” pour inverser
- **p (La Prédiction)** : C'est le vecteur (256D) produit par l'agent.
- **a^+ (Le Positif / La Cible)** : C'est l'embedding du prompt qui a obtenu la meilleure récompense dans le batch mémoire
- **a_j^- (Les Négatifs)** : Ce sont les embeddings de tous les autres 63 prompts.
- **sim** : C'est le produit scalaire (ou similarité cosinus).
- **τ (La Température)**: Il contrôle la “netteté” de distribution de la sélection. $\tau = 0,1$.

Annexe – value loss

Role :

- force le critique à prédire à l'avance la récompense qu'une phrase va obtenir

$$L_{\text{value}} = \frac{1}{B} \sum_i (V(s_i) - r_i)^2$$

- **B(Batch Size)** : le lot de **64**.
- **s_i (L'État)** : C'est l'entrée du réseau (l'embedding 1024D).
- **\$V(s_i)\$** : La Prédiction de la récompense.
- **\$r_i\$ (La Réalité / La Cible)** : C'est la récompense réelle (1 - CER).
- Cela aide Pour réduire la variance et stabiliser l'apprentissage.

Annexe – Entropie

Rôle : empêcher le "Policy Collapse", forcer la variété, ne pas être trop sûr trop vite.

$$H = \overline{\sigma_{\text{dim}}(\hat{a})} + 0,5 \cdot \overline{\text{pdist}(\hat{a})}$$

- **Pdist** : la distance moyenne entre vecteur batch.
- **Variance** 64 phrases du batch en ligne et 256 dim en Colonne.

Annexe – Pénalité

$$\text{Pen}_{\text{div}} = \frac{1}{B(B-1)} \sum_{i \neq j} \left(\frac{\hat{a}_i \cdot \hat{a}_j}{\|\hat{a}_i\| \|\hat{a}_j\|} \right)^2$$

- B (Batch Size)
- $\hat{a}_i \cdot \hat{a}_j$ (Produit Scalaire) : C'est la mesure de l'angle.

Annexe – Replay buffer

Définition : C'est une mémoire tampon qui stocke les dernières expériences sous forme de tuples : (phrase, prompt choisi, récompense obtenue).

Rôle :

- Stabiliser l'apprentissage: perte moyenne sur un batch de 64.
- Permet de Calculer l'Avantage A pour la loss de policy

Annexe – ϵ -greedy

Définition : règle de décision simple qui intervient au moment de choisir le prompt

- **Exploitation ($1 - \epsilon$)** : L'agent choisit l'action qu'il pense être la meilleure (celle avec le score le plus haut)
- **Exploration (ϵ)** : L'agent ignore son intuition et choisit une action totalement au hasard.
- **Comportement** : 0.5 au debut, descent exponentielle (0.05% /600)

ϵ -greedy Vs Entropie

- **ϵ -greedy (Intervention Externe)** : Force l'exploration en imposant parfois un choix aléatoire, ignorant la confiance du modèle.
- **Entropie (Régularisation Interne)** : Terme de la Loss qui pénalise mathématiquement les certitudes trop rapides pour garder les probabilités "étaillées"

Annexe – Normalisation adaptative des récompenses

- Définition : Une technique statistique qui transforme la récompense brute (le score CER entre 0 et 1) en une valeur relative appelée Avantage

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + 10^{-8}}$$

- **r_i** : La récompense de la phrase actuelle (1 - CER).
- **Moyenne** : La moyenne des récompenses dans le Replay Buffer (64)
- **Écart-type** : L'écart-type des récompenses..

Annexe – FAISS

- Définition : Une technique statistique qui transforme la récompense brute (le score CER entre 0 et 1) en une valeur relative appelée Avantage

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r + 10^{-8}}$$

- **r_i** : La récompense de la phrase actuelle (1 - CER).
- **Moyenne** : La moyenne des récompenses dans le Replay Buffer (64)
- **Écart-type** : L'écart-type des récompenses..